

Lab3：中断与中断处理流程 实验报告

南开大学 计算机学院和密码与网络空间安全学院

年级：2023 级

专业：信息安全

姓名及学号：于昊汐（2312642）、陈安琪（2312481）、孙丽菲（2312724）

日期：2025 年 10 月 26 日

一、实验目的

- 理解 RISC-V 中断与异常概念：明确中断与异常的区别及统一处理模型，掌握特权级（M/S/U）和中断委托机制的作用。
- 掌握中断处理全流程：熟悉从硬件触发到软件处理的完整链路，包括上下文保存、中断分发、处理及恢复的实现逻辑。
- 实现并验证时钟中断：完成时钟中断初始化、定时设置及处理（计数、打印、关机），验证其周期性触发与响应。
- 掌握异常处理方法：实现非法指令、断点等异常的处理，包括原因解析、信息打印及返回控制。
- 理解上下文保存与恢复机制：分析寄存器与 CSR 的保存 / 恢复逻辑，明确其与 TrapFrame 的对应关系及对程序续行的必要性。
- 关联中断与 OS 核心功能：建立中断机制与进程调度、设备管理等 OS 功能的关联，为后续学习奠定基础。

二、实验内容

- 搭建中断上下文框架：完成 trapentry.S 中 _alltraps 和 trapret 的汇编逻辑，用 SAVE_ALL/RESTORE_ALL 宏将寄存器及 sepc/scause 等保存到 TrapFrame，实现上下文保存与恢复。
- 完善时钟中断处理：在 interrupt_handler 中补全时钟中断逻辑，设置下一次中断，计数 ticks，每 100 次打印信息，累计 10 次后关机，验证时钟中断功能。
- 解析中断核心机制：解答 move a0, sp 的作用、SAVE_ALL 寄存器顺序与 TrapFrame 的对应关系、保存所有寄存器的必要性。
- 分析 CSR 操作逻辑：解释 sscratch 相关指令的作用，以及 stval/scause 只保存不恢复的原因。
- 实现异常处理：在 exception_handler 中处理非法指令和断点异常，打印类型与地址，调整 sepc 确保程序续行。
- 验证功能：运行系统，测试时钟中断的定时打印、关机功能及异常处理效果，确认机制正确性。

三、实验过程

练习 1：完善中断处理（需要编程）

请编程完善 trap.c 中的中断处理函数 trap，在对时钟中断进行处理的部分填写 kern/trap/trap.c 函数中处理时钟中断的部分，使操作系统每遇到 100 次时钟中断后，调用 print_ticks 子程序，向屏幕上打印一行文字“100 ticks”，在打印完 10 行后调用 sbi.h 中的 shut_down () 函数关机。

要求完成问题 1 提出的相关函数实现，提交改进后的源代码包（可以编译执行），并在实验报告中简要说明实现过程和定时器中断处理的流程。实现要求的部分代码后，运行整个系统，大约每 1 秒会输出一行“100 ticks”，输出 10 行。

设计实现

```
#include <sbi.h>
```

首先，添加包含头文件。根据功能实现要求，需要在打印 10 行后调用 sbi_shutdown () 关机，而 sbi_shutdown () 函数声明在 sbi.h 中，没有此头文件将会导致编译错误：“undefined reference to sbi_shutdown”。

SBI 的作用：1.SBI 是 RISC-V 的机器模式 (M-mode) 与监管模式 (S-mode) 之间的标准接口；2.SBI 提供系统级服务：定时器设置、控制台 I/O、系统关机等，其中 sbi_shutdown () 是通过 SBI 调用机器模式的关机服务。

```
static int num = 0; // 跟踪打印次数
```

然后，添加静态变量 num。根据需求，需要跨多次中断记录已打印的次数，static 能确保该变量在程序运行期间始终存在，不会因函数执行结束而被销毁，从而持续保留计数状态；并且，静态变量可以将其作用域限定在当前文件内，避免与其他变量发生命名冲突。

相较于全局变量，静态变量的封装性更好，限制访问范围，可减少被意外修改的可能；也能让相关数据与处理逻辑更紧密地集中在同一文件中，符合模块化设计原则。

```
static void print_ticks() {
    cprintf("%d ticks\n", TICK_NUM);
    num++; // 新添加
#ifdef DEBUG_GRADE
    cprintf("End of Test.\n");
    panic("EOT: kernel seems ok.");
#endif
}
```

接下来，在 print_ticks () 函数中添加 num++。用来记录每次打印次数，每打印一次 100 ticks，静态变量 num 数值加一。

print_ticks () 函数的作用是定时打印系统当前的时钟计数（tick 数），记录被调用次数，并在调试模式（DEBUG_GRADE）下输出测试结束信息并终止内核执行。

```
case IRQ_S_TIMER:

    // (1) 设置下次时钟中断
    clock_set_next_event();

    // (2) 计数器 (ticks) 加一
    ticks++;

    // (3) 当计数器加到100时，调用 print_ticks() 输出并增加打印次数
    if (ticks >= TICK_NUM) {
        print_ticks();
    }
```

```

        ticks = 0; // 重置计数器

// (4) 判断打印次数，当打印次数为10时，调用关机函数
        if (num >= 10) {
            sbi_shutdown();
        }
    }
    break;

```

最后，补充 `interrupt_handler()` 函数，实现时钟中断处理逻辑。在 `IRQ_S_TIMER` 分支中完成时钟中断处理：

(1) 设置下次时钟中断：调用 `clock_set_next_event()` 函数。其中 `clock_set_next_event()` 函数定义在 `kern/driver/clock.c` 文件中。

```

void clock_set_next_event(void) {
    sbi_set_timer(get_cycles() + timebase);
}

```

在 `clock_set_next_event()` 函数中，实现流程为：首先，通过 `get_cycles()` 函数获取当前 CPU 时钟周期数；然后，加上时间间隔 `timebase`，表示下次中断距离当前的周期数；最后，调用 `sbi_set_timer()` 函数，通过 SBI 设置定时器，在指定的周期数到达时触发中断。

(2) 设置 `ticks++`，递增全局中断计数器。`ticks` 是 `clock.c` 中的全局变量。

```

volatile size_t ticks;

```

变量 `ticks` 被 `volatile` 关键字修饰，这是因为该变量会在中断中被修改，`volatile` 能够防止编译器对其进行优化，确保每次访问都能获取到最新的值，从而保证全局状态的正确性。

(3) 当 `ticks >= 100` 时：调用 `print_ticks()` 输出“100 ticks”，`print_ticks()` 函数内部会将 `num++`；重置 `ticks=0`。

其中，设置为 `>=TICK_NUM` 的原因是：防止因为某种异常导致 `ticks` 的数值跳过 100 的情况，确保 `>=` 的条件一定被触发，避免无限等待。

重置 `ticks = 0` 的原因是：每 100 次中断为一个周期，重置后开始新周期，避免 `ticks` 无限增长。

(4) 判断打印次数 `num >= 10` 时，调用 `sbi_shutdown()` 函数执行关机。

在 `print_ticks()` 后检查关机条件，确保十次“100 ticks”成功打印后再执行关机操作。

最后，执行 `make` 和 `make qemu` 命令，编译代码并运行内核，实现效果如图所示。

```

sunny@sunny-VMware-Virtual-Platform:~/桌面/lab3$ make
+ cc kern/init/entry.S
+ cc kern/init/init.c
+ cc kern/libs/stdio.c
+ cc kern/debug/kdebug.c
+ cc kern/debug/kmonitor.c
+ cc kern/debug/panic.c
+ cc kern/driver/clock.c
+ cc kern/driver/console.c
+ cc kern/driver/dtb.c
+ cc kern/driver/intr.c
+ cc kern/trap/trap.c
+ cc kern/trap/trapentry.S
+ cc kern/mm/best_fit_pmm.c
+ cc kern/mm/default_pmm.c
+ cc kern/mm/pmm.c
+ cc libs/printfmt.c
+ cc libs/readline.c
+ cc libs/sbi.c
+ cc libs/string.c
+ ld bin/kernel
riscv64-unknown-elf-objcopy bin/kernel --strip-all -O binary bin/ucore.img

```

```

sunny@sunny-VMware-Virtual-Platform:~/桌面/lab3$ make qemu

```

```

OpenSBI v0.4 (Jul  2 2019 11:53:53)

```

```

      _ _ _ _ _      _ _ _ _ _      _ _ _ _ _
 / _ _ \      / _ _ \      / _ _ \      / _ _ \
| | | | | _ _ _ _ _ | ( _ _ | | ) | | | | | | |
| | | | | ' _ \ / _ \ ' _ \ \ _ \ | | _ < | |
| | | | | |_) | _/ | | |_) | |_) | | |
 \ _ _/ | . _/ \ _ _/ | | _ _/ | | _ _/ | |
      | |
      | |

```

```

Platform Name       : QEMU Virt Machine
Platform HART Features : RV64ACDFIMSU
Platform Max HARTs   : 8
Current Hart        : 0
Firmware Base       : 0x80000000
Firmware Size       : 112 KB
Runtime SBI Version  : 0.1

```

```

PMP0: 0x0000000080000000-0x000000008001ffff (A)
PMP1: 0x0000000000000000-0xffffffffffffff (A,R,W,X)
DTB Init
HartID: 0
DTB Address: 0x82200000
Physical Memory from DTB:
  Base: 0x0000000080000000
  Size: 0x0000000080000000 (128 MB)
  End:  0x0000000087ffffff
DTB init completed

```

```

(THU.CST) os is loading ...
Special kernel symbols:
  entry 0xffffffffc0200054 (virtual)
  etext 0xffffffffc0201f2c (virtual)
  edata 0xffffffffc0206028 (virtual)
  end   0xffffffffc02064a0 (virtual)
Kernel executable memory footprint: 26KB
memory management: default_pmm_manager
physical memory map:
  memory: 0x0000000008000000, [0x0000000080000000, 0x0000000087ffffff].
check_alloc_page() succeeded!
satp virtual address: 0xffffffffc0205000
satp physical address: 0x0000000080205000
++ setup timer interrupts
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
sunyy@sunyy-VMware-Virtual-Platform:~/桌面/lab3$

```

从输出结果来看，成功实现每到 100 次时钟中断后，调用 print_ticks 子程序，向屏幕上打印一行文字“100 ticks”，在打印完 10 行后调用 sbi.h 中的 shut_down () 函数关机。

定时器中断处理的流程

第一步：定时器触发（SBI 硬件中断）

在硬件层面发生：机器模式定时器 → 硬件中断信号 → CPU 响应中断

具体过程为：

1. 定时器设置：之前通过 sbi_set_timer (get_cycles () + timebase) 函数设置下一次时间中断；
2. 时间到达：当 CPU 周期数达到设定值时，硬件产生中断信号；
3. 中断类型：产生的中断类型为 Supervisor Timer Interrupt (IRQ_S_TIMER = 5)；
4. 硬件响应：CPU 检测到中断待决位，准备处理中断。

第二步：CPU 跳转到 __alltraps 保存现场

首先，硬件自动操作：pc = stvec # 跳转到异常向量地址。其中，stvec 寄存器的设置在 idt_init () 函数中。

```
write_csr(stvec, &__alltraps);
```

其作用为：将 __alltraps 标签的地址写入 stvec 寄存器中。&__alltraps 为中断处理入口的地址。

现场保存的详细过程为：把被打断的 CPU 状态保存到栈上，把栈指针当作参数传给高级语言的 trap 处理函数，然后跳转调用该处理函数。

```

__alltraps:
    SAVE_ALL
    move  a0, sp
    jal  trap

```

其中，SAVE_ALL 宏的工作：完成了把当前 CPU 的全部寄存器和必要的控制寄存器值保存到栈上。

```

.macro SAVE_ALL
    csrw sscratch, sp          # 临时保存当前栈指针
    addi sp, sp, -36 * REGBYTES # 分配trapframe空间

    # 保存所有通用寄存器 (31个)
    STORE x0, 0*REGBYTES(sp)   # zero寄存器
    STORE x1, 1*REGBYTES(sp)   # ra (返回地址)
    # ... 保存 x3-x31

    # 保存控制状态寄存器
    csrrw s0, sscratch, x0     # 获取原sp, 同时清零sscratch
    csrr s1, sstatus           # 状态寄存器
    csrr s2, sepc              # 异常程序计数器
    csrr s3, sbadaddr          # 错误地址
    csrr s4, scause            # 异常原因

    # 将CSR值存入trapframe
    STORE s0, 2*REGBYTES(sp)   # 原栈指针
    STORE s1, 32*REGBYTES(sp)  # status
    STORE s2, 33*REGBYTES(sp)  # epc
    STORE s3, 34*REGBYTES(sp)  # badvaddr
    STORE s4, 35*REGBYTES(sp)  # cause
.endm

```

所以，完整的保存现场流程为：首先，在初始化时设置 stvec 寄存器的内容为 __alltraps 标签的地址；然后，在中断发生时，硬件自动执行 pc=stvec，并保存被中断的指令地址、中断使能状态，设置中断原因等信息；最后，CPU 执行跳转后的代码，保存现场。

第三步：调用 trap () → trap_dispatch ()

在保存现场后，将 trapframe 指针作为参数，跳转到 trap () 函数。

```

void trap(struct trapframe *tf) {
    // dispatch based on what type of trap occurred
    trap_dispatch(tf);
}

static inline void trap_dispatch(struct trapframe *tf) {
    if ((intptr_t)tf->cause < 0) {
        // interrupts
        interrupt_handler(tf);
    } else {
        // exceptions
        exception_handler(tf);
    }
}

```

在 trap_dispatch () 函数中，区分一个 trap 是“中断 (interrupt)”还是“异常 (exception)”，然后把处理权分派给相应的处理函数。判断方法是把 tf->cause 视为有符号整数并检查其符号位（负号表示“中断”）。

对于时钟中断，tf->cause = 0x8000000000000005，(intptr_t) tf->cause < 0 为真，进入 interrupt_handler (tf) 函数。

第四步：interrupt_handler () 识别 IRQ_S_TIMER

```
void interrupt_handler(struct trapframe *tf) {
    intptr_t cause = (tf->cause << 1) >> 1; // 清除最高位
    switch (cause) {
```

在 interrupt_handler() 函数中，先从 tf->cause 中去掉最高位（MSB，即“中断”标志），得到实际的中断编号，然后用 switch 根据该编号分派具体的中断处理逻辑。

在 RISC-V 的约定中，scause 的最高位用来表示“这是中断（1）还是异常（0）”，低位是具体的中断 / 异常号。

对于时钟中断，原始 cause=0x8000000000000005，左移一位后为 0x000000000000000a，再右移一位得到 0x0000000000000005，为 IRQ_S_TIMER。

IRQ_S_TIMER 类型中断处理如下：

```
case IRQ_S_TIMER:
    // 时钟中断处理逻辑
    clock_set_next_event();
    ticks++;
    if (ticks >= TICK_NUM) {
        print_ticks();
        ticks = 0;
        if (num >= 10) {
            sbi_shutdown();
        }
    }
    break;
```

第五步：设置下次中断并递增 ticks

首先，设置下一次中断：

```
void clock_set_next_event(void) { sbi_set_timer(get_cycles() + timebase); }
```

详细的执行过程为：

1.get_cycles(): 读取当前 CPU 周期数

```
static inline uint64_t get_cycles(void) {
    uint64_t n;
    __asm__ __volatile__("rdtime %0" : "=r"(n)); // RISC-V rdtime指令
    return n;
}
```

通过一个 GCC 内联汇编指令 rdtime 从 RISC-V 的时间寄存器读取当前时间 / 时钟计数（返回到 uint64_t n），并把它作为 64 位无符号整数返回。static inline 表明编译器会把函数内联（如果合适），不会产生外部符号。

2.计算下次中断时间：

```
next_time = current_cycles + 100000 // timebase = 100000
```

下次中断时间等于当前执行时间周期数加上时间间隔 timebase。

3.SBI 调用：

```
void sbi_set_timer(unsigned long long stime_value) {
    sbi_call(SBI_SET_TIMER, stime_value, 0, 0);
}
```

sbi_set_timer 是操作系统在 S 模式下请求固件 / 监控设置下一次定时器中断的接口；它把目标时间 stime_value 交给固件 / 平台，让平台在对应的时间触发一个定时中断。

然后，递增计数器：ticks++；。

第六步：累计满 100 次则打印并重置计数

首先，进行条件判断：

```
if (ticks >= TICK_NUM) { // TICK_NUM = 100
```

当 ticks 的值大于等于 100 时，则执行打印操作。

打印操作：

```
static void print_ticks() {
    cprintf("%d ticks\n", TICK_NUM); // 输出 "100 ticks"
    num++;                          // 增加打印计数
}
```

最后，对计数器进行重置：ticks=0；，重新开始计数。

第七步：打印 10 次后调用 sbi_shutdown () 关机

首先，需要进行关机条件的判断：

```
if (num >= 10) {
    sbi_shutdown();
}
```

当打印次数大于等于 10 的时候，才调用 sbi_shutdown () 函数进行关机操作。

关机实现：

```
void sbi_shutdown(void) {
    sbi_call(SBI_SHUTDOWN, 0, 0, 0);
}
```

sbi_shutdown () 函数是操作系统在 S 模式下调用 SBI 来请求平台或固件关机的接口。它通过 sbi_call 发出请求，让底层固件执行实际的电源关闭或平台复位操作。

SBI 调用机制：

```
uint64_t sbi_call(uint64_t sbi_type, uint64_t arg0, uint64_t arg1, uint64_t
arg2) {
    uint64_t ret_val;
    __asm__ volatile (
        "mv x17, %[sbi_type]\n" // 功能号放入x17
        "mv x10, %[arg0]\n"     // 参数放入x10-x12
        "mv x11, %[arg1]\n"
        "mv x12, %[arg2]\n"
        "ecall\n"               // 调用机器模式
```

```

        "mv %[ret_val], x10" // 获取返回值
        : [ret_val] "=r" (ret_val)
        : [sbi_type] "r" (sbi_type), [arg0] "r" (arg0), [arg1] "r" (arg1),
[arg2] "r" (arg2)
        : "memory"
    );
    return ret_val;
}

```

sbi_call() 函数是 RISC-V 操作系统调用 SBI 的通用接口，通过 ecall 指令向底层固件或机器模式请求服务（如设置 timer、关机、发送 IPI 等）。

该函数实现功能：

1. 把 SBI 功能号和参数放到指定寄存器（x17、x10~x12）。
2. 执行 ecall 进入机器模式（M-mode 或固件环境）。
3. 固件执行相应操作后，将返回值放回 x10。
4. C 代码读取返回值并返回给调用者。

第八步：恢复现场，返回被中断的程序

中断处理完成后：

```

__trapret:
    RESTORE_ALL    # 恢复所有寄存器
    sret           # 从S模式返回

```

__trapret 是中断处理完成的出口，通过 RESTORE_ALL 宏恢复 CPU 状态，再用 sret 返回原来的程序继续执行。

其中，RESTORE_ALL 宏的工作：把保存的寄存器值重新加载到对应寄存器；恢复中断发生前的上下文。

```

.macro RESTORE_ALL
    LOAD s1, 32*REGBYTES(sp) # 加载status
    LOAD s2, 33*REGBYTES(sp) # 加载epc

    csrw sstatus, s1          # 恢复状态寄存器
    csrw sepc, s2             # 恢复程序计数器

    # 恢复所有通用寄存器
    LOAD x1, 1*REGBYTES(sp)  # ra
    # ... 恢复x3-x31
    LOAD x2, 2*REGBYTES(sp)  # 最后恢复sp
.endm

```

sret 为 RISC-V 特权指令，用于从模式返回到 trap 发生前的特权模式。其实现功能为：

1. 从 sepc 寄存器中获取返回地址（被中断的指令）；
2. 将 sstatus.SIE（中断使能位）恢复；
3. 恢复 trap 前的特权级；
4. 跳转回 trap 前的指令执行位置。

整体的流程可以表示为：

```
时间轴： ---[中断]---[保存]---[处理]---[恢复]---[继续]---
硬件：   定时器→CPU中断
汇编：   __alltraps→SAVE_ALL
C代码：   trap()→interrupt_handler()
汇编：   RESTORE_ALL→sret
程序：   继续执行
```

扩展练习 Challenge1：描述与理解中断流程

回答：描述 ucore 中处理中断异常的流程（从异常的产生开始），其中 `mov a0, sp` 的目的是什么？
`SAVE_ALL` 中寄存器保存在栈中的位置是什么确定的？对于任何中断，`__alltraps` 中都需要保存所有寄存器吗？请说明理由。

ucore 中处理中断 / 异常的完整流程（从异常产生开始）

整体而言，ucore 的中断异常处理流程为硬件触发→汇编保存→C 层分发→汇编恢复，具体流程如下：

1、异常 / 中断产生

触发场景：

外部中断（如时钟计时结束、串口收数据）：外设向 CPU 发送中断信号，硬件自动识别并标记中断类型；

内部异常（如非法指令、断点、缺页）：CPU 执行指令时检测到错误，自动记录异常原因；

主动陷入（如 `ecall` 系统调用、`ebreak` 断点）：程序主动执行特权指令，触发 Trap。

硬件自动操作，保存现场信息到 CSRs：

`sepc` ← 当前 PC（被中断的指令地址）

`scause` ← 中断 / 异常原因编码

`stval` ← 附加信息（如故障地址）

`sstatus` ← 保存中断使能状态（`SIE`→`SPIE`）

最后切换特权级，并跳转到 `stvec` 寄存器指向的中断入口（即 `__alltraps`）。

2、汇编层上下文保存（`__alltraps` 与 `SAVE_ALL`）

硬件跳转到 `kern/trap/trapentry.S` 的 `__alltraps` 后，首先执行 `SAVE_ALL` 宏，完成当前 CPU 状态快照：

步骤 1：把当前栈指针 `sp` 暂存到 `sscratch` 寄存器，这是区分内核态 / 用户态中断的关键；

步骤 2：调整栈指针，在栈上开辟 36 个寄存器大小的空间，用于存储 `struct trapframe`；

步骤 3：按顺序保存 32 个通用寄存器到栈，其中 `x0`、`x1`、`x3-x31` 直接保存，`x2` (`sp`) 通过 `sscratch` 读取后保存；

步骤 4：读取关键 CSR（`sstatus/sepc/sbadaddr/scause`）到通用寄存器，再保存到栈的对应位置，最终在栈上构建出完整的 `struct trapframe` 结构体。

步骤 5：把栈顶指针 `sp` 即 `struct trapframe` 的起始地址传给 `a0` 寄存器，为调用 C 层处理函数 `trap()` 做准备；

步骤 6：`jal trap` → 跳转到 C 语言实现的 `trap()` 函数，进入中断 / 异常的逻辑处理。

3、C 层中断分发与处理（`trap.c`）

接收 `a0` 传递的中断上下文指针，根据 `tf->cause` 判断类型：

若 `(intptr_t) tf->cause < 0` (`scause` 最高位为 1) → 外部中断, 调用 `interrupt_handler(tf)`;

若 `(intptr_t) tf->cause ≥ 0` (`scause` 最高位为 0) → 内部异常, 调用 `exception_handler(tf)`。

4. 汇编层上下文恢复与返回 (`__trapret` 与 `RESTORE_ALL`)

C 层处理完成后, 回到 `_alltraps` 后续的 `trapret` 标签, 执行 `RESTORE_ALL` 宏恢复状态:

步骤 1: 从栈中读取 `sstatus/sepc` 的值, 写回对应 CSR;

步骤 2: 按保存逆序恢复 32 个通用寄存器;

步骤 3: 执行 `sret` 指令, 硬件自动完成:

把 `sepc` 的值赋给程序计数器 `pc`, 跳回被中断的指令地址;

根据 `sstatus.SPP` 位恢复特权级;

恢复中断使能状态, 回到被中断的程序继续执行。

`move a0, sp` 的目的是什么?

我们观察这段代码:

```
__alltraps:
    SAVE_ALL
    move a0, sp
    jal trap
```

我们先保存了所有寄存器, 此时 `sp` 指向栈顶, 也就是刚保存的 `trapframe` 结构体的起始地址, 而 `a0` 是 RISC-V 中第一个函数参数寄存器, 这样 `trap (struct trapframe *tf)` 就能访问到所有保存的寄存器信息

所以这句代码的目的是, 按照 RISC-V 调用约定, 将 `trapframe` 结构体的指针作为参数传递给 C 函数 `trap()`。

`SAVE_ALL` 中寄存器保存在栈中的位置是如何确定的?

位置由 `trapframe` 结构体的内存布局和 `REGBYTES` 常量共同确定:

```
// trap.h 中定义的结构体布局
struct trapframe {
    struct pushregs gpr; // 32个通用寄存器, 占 0-31*REGBYTES
    uintptr_t status;    // 32*REGBYTES
    uintptr_t epc;       // 33*REGBYTES
    uintptr_t badvaddr;  // 34*REGBYTES
    uintptr_t cause;     // 35*REGBYTES
};
```

在 `SAVE_ALL` 中:

```
addi sp, sp, -36 * REGBYTES # 在栈上分配 trapframe 空间
STORE x0, 0*REGBYTES(sp)    # x0 保存在偏移 0 处
STORE x1, 1*REGBYTES(sp)    # x1 保存在偏移 1*REGBYTES 处
# ... 以此类推
```

那么在编译时通过结构体字段的偏移量计算, 可以保证汇编代码中的保存位置与 C 结构体定义完全对应。

对于任何中断，__alltraps 中都需要保存所有寄存器吗？请说明理由。

需要保存所有寄存器，理由如下：

- 1、在进入 trap () 之前，无法预知具体是哪种中断 / 异常，也不知道处理程序会用到哪些寄存器。为了安全起见，必须保存全部上下文。
- 2、中断可能发生在任何代码位置，被中断的程序可能正在使用任何寄存器。为了能够正确返回到被中断点，必须完整保存所有寄存器状态。
- 3、如果中断来自用户态，需要完整的上下文保存来实现用户态 - 内核态的切换。
- 4、不同的中断处理程序可能需要访问不同的寄存器信息。比如系统调用需要参数寄存器，页错误需要访问出错的地址等。
- 5、统一保存所有寄存器比按需保存更简单可靠，避免了因遗漏保存导致的难以调试的问题。

例外情况：理论上某些特定中断可能不需要所有寄存器，但为了保持代码的简洁性和可靠性，ucore 采用了 "一刀切" 的策略，对所有 trap 都保存完整上下文。

扩展练习 Challenge2：理解上下文切换机制

回答：在 trapentry.S 中汇编代码 `csrw sscratch, sp; csrrw s0, sscratch, x0` 实现了什么操作，目的是什么？`save all` 里面保存了 `stval` `scause` 这些 csr，而在 `restore all` 里面却不还原它们？那这样 store 的意义何在呢？

csrw sscratch, sp; csrrw s0, sscratch, x0 的操作和目的

这两条指令的核心作用是安全保存中断发生前的栈指针（旧 sp），同时隐含区分中断来源（内核态 / 用户态），为后续上下文保存和恢复提供基础。

1、操作本身

`csrw sscratch, sp`：将中断发生时的栈指针写入 `sscratch` 寄存器，完成对旧 sp 的临时寄存。

`csrrw s0, sscratch, x0`：这是一条读 - 写交换指令，先读取 `sscratch` 中暂存的旧 sp 并保存到 `s0` 寄存器，再将 `x0` 写入 `sscratch` 以重置该寄存器。

2、核心目的

保存旧栈指针：旧 sp 是中断上下文的关键部分。通过 `sscratch` 中转后，`s0` 中存储的旧 sp 会被最终写入 `struct trapframe` 的对应位置，确保中断返回时能恢复原栈指针，让程序继续正常执行。

隐含区分中断来源：结合 `sscratch` 的初始约定（内核态下初始为 0，用户态下初始为内核栈地址），通过 `s0` 中旧 sp 的地址范围（内核栈或用户栈），可间接判断中断发生在核心态还是用户态，为后续处理提供依据。

为什么 SAVE_ALL 保存了 stval、scause 等 CSR，但 RESTORE_ALL 不还原它们？

观察：在 `SAVE_ALL` 中我们保存了：

```
csrr s1, sstatus    # 保存到32*REGBYTES(sp)
csrr s2, sepc       # 保存到33*REGBYTES(sp)
csrr s3, sbadaddr   # 保存到34*REGBYTES(sp)
csrr s4, scause     # 保存到35*REGBYTES(sp)
```

而在 `RESTORE_ALL` 中只恢复了：

```

LOAD s1, 32*REGBYTES(sp)
LOAD s2, 33*REGBYTES(sp)
csrw sstatus, s1      # 只恢复sstatus
csrw sepc, s2         # 只恢复sepc
# stval(scause)没有恢复!

```

为了回答这个问题，我们可以从这些寄存器中保存的信息区别入手，来分析它们的作用：

1、诊断和调试：

scause：记录中断 / 异常的原因，用于 trap_dispatch() 分发处理

stval：提供附加信息（如故障地址、非法指令内容）

这些信息对处理中断至关重要，但对返回不重要

2、处理逻辑需要：

在 C 代码的 interrupt_handler() 和 exception_handler() 中，需要根据 tf->cause 来判断中断类型

需要 tf->badvaddr 来处理页错误等异常

3、这些 CSR 的特性：

scause、stval 是只写一次的寄存器 - 硬件在陷入时设置，软件读取使用

它们不代表 "程序状态"，而是 "事件信息"

返回时硬件会自动清除或重新设置这些寄存器

4、sstatus 和 sepc 为什么需要恢复：

sstatus：包含中断使能位 (SPP、SIE、SPIE 等)，影响后续执行环境

sepc：决定返回到哪里执行，是程序执行流的关键

所以我们来总结一下这个问题的答案：

需要恢复的：影响程序持续执行状态的寄存器（通用寄存器、sstatus、sepc、sp）

不需要恢复的：记录单次事件信息的寄存器（scause、stval）

store 的意义：为中断处理程序提供必要的上下文信息来进行正确的分发和处理，即使这些信息在返回时不需要恢复。

扩展练习 Challenge3：完善异常中断

编程完善触发一条非法指令异常 mret 和 ebreak，在 kern/trap/trap.c 的异常处理函数中捕获，并对其进行处理，简单输出异常类型和异常指令触发地址，即 "Illegal instruction caught at 0x (地址)"，"ebreak caught at 0x (地址)" 与 "Exception type:Illegal instruction", "Exception type: breakpoint"。

首先我们找到对应的非法指令异常处理部分的代码并进行完善，结果如下：

```

case CAUSE_ILLEGAL_INSTRUCTION:
    // 非法指令异常处理
    /* LAB3 CHALLENGE3 YOUR CODE : */
    /* (1)输出指令异常类型 ( Illegal instruction)
    * (2)输出异常指令地址
    * (3)更新 tf->epc寄存器
    */

```

```
//(1)输出输出指令异常类型 ( Illegal instruction)
cprintf("Exception: Illegal instruction\n");
//(2)输出异常指令地址
cprintf("Illegal instruction caught at 0x%08x\n", tf->epc);
//(3)更新 tf->epc寄存器 (跳过非法指令, 避免再次陷入)
tf->epc += 4;

break;
```

这里的处理分为以下三步:

- 1、输出异常类型, 表明当前捕获到的异常属于非法指令类型;
- 2、输出异常触发指令的地址, 也就是 CPU 执行出错的那条指令所在位置, 能帮助定位问题发生点, 我们首先将其存在了寄存器 sepc 中的值, 然后由于要进行中断异常处理, 保存上下文, 该寄存器的值又保存在了 tf->epc 中, 所以这里直接输出 tf->epc 的值做为异常触发指令的地址;
- 3、修正 tf->epc, 跳过异常指令, 由于 RISC-V 指令长度为 4 字节 (32 位), 我们将 tf->epc 的值加 4, 意味着让处理器在异常返回后, 从下一条指令继续执行, 而不是重新执行那条非法指令。这样可以避免系统反复陷入相同异常导致死循环。

接着, 我们编写断点异常的处理逻辑, 如下所示

```
case CAUSE_BREAKPOINT:
    //断点异常处理
    /* LAB3 CHALLENGE3 YOUR CODE : */
    /*(1)输出指令异常类型 ( breakpoint)
    *(2)输出异常指令地址
    *(3)更新 tf->epc寄存器
    */

    //(1)输出指令异常类型 ( breakpoint)
    cprintf("Exception type: breakpoint\n");
    //(2)输出异常指令地址
    cprintf("ebreak caught at 0x%08x\n", tf->epc);
    //(3)更新 tf->epc寄存器 (跳过断点, 继续执行后面的指令)
    tf->epc += 2;

    break;
```

这部分同样也是分为三步:

- 1、输出异常类型, 提示当前捕获到的异常属于断点类型, 说明程序主动触发了调试陷入;
- 2、输出异常指令地址, 与上面非法指令异常处理类似, 还是 tf->epc 的值;
- 3、修正 tf->epc, 跳过断点指令, 由于 ebreak 本身只是一个用于调试的指令, 如果在返回后仍从同一地址继续执行, 则会再次触发异常, 形成死循环。我们将 tf->epc 加 2, 指向下一条指令, 使得系统得以安全继续运行。

将 tf->epc 加 2 而不是加 4 的原因如下:

其实这里我们一开始的做法仍然是和上面非法指令异常一样, 通过将程序计数器加 4 使异常返回后直接从下一条指令开始执行, 但后来运行 make qemu 发现程序一直在输出, 似乎进入了死循环, 我们推断可能是我们重新设置的指令是不正确的, 修改了很久, 后来查阅相关资料发现在 risc-v 中可能会启用压缩指令, 当编译器或汇编器启用了 C 扩展 (例如 -march=rv32imac), ebreak 实际上可能被汇编成压缩版本 c.ebreak, 它的机器码长度就是 2 字节, 而不是 4 字节, 于是我们尝试将 tf->epc 改为加 2, 惊

喜地发现一开始的死循环消失啦，输出是正常的！这也进一步验证了我们当前的 RISC-V 环境确实启用了压缩指令。

为了验证我们编写的异常处理逻辑的正确性，我们定义一个测试函数，试图分别执行一条非法指令异常 mret 和 ebreak，触发异常，从而测试我们的异常处理逻辑。

```
static void test_exceptions(void) {
    printf("\n--- Triggering illegal instruction test ---\n");
    asm volatile("mret"); // 在非 M 模式下会触发非法指令异常
    printf("\n--- Triggering ebreak test ---\n");
    asm volatile("ebreak"); // 触发断点异常
}
```

我们编写具体的异常处理函数如上所示，首先打印非法指令异常测试的提示信息，然后执行 mret 指令，由于当前处于 S 模式的，而 mret 仅限在 M 模式执行，因此会触发一次非法指令异常，进入中断处理程序处理（即由 stvec 指向的 `_alltraps`），trap 框架首先会把上下文保存到 trapframe 并把 `tf->cause` 传入 `trap_dispatch`，然后跳转到 `exception_handler` 的 `CAUSE_ILLEGAL_INSTRUCTION` 分支进行具体处理，先打印异常类型和触发地址（`tf->epc`，即 mret 的地址），然后通过 `tf->epc += 4` 跳过那条触发异常的指令；这样返回时 CPU 会从 mret 之后的下一条指令继续执行，防止再次执行 mret 导致无限陷入。

异常处理完成并返回后，继续打印第二条提示并执行 ebreak 指令，从而触发断点异常。同样类似上面的逻辑跳转到 `exception_handler` 的 `CAUSE_BREAKPOINT` 分支进行相应的处理，同样是打印异常类型与 `tf->epc`（ebreak 指令地址），再用 `tf->epc += 2` 跳过该指令，以保证返回后能从断点后的下一条指令继续执行，避免反复进入断点处理。

```
if (num >= 10) {
    test_exceptions(); // 调用异常测试函数
    sbi_shutdown();
}
```

最后，我们在完成 10 次时钟中断之后调用该函数，函数执行结束后，再调用 `sbi_shutdown()` 执行关机操作。

代码编写完成后，我们执行 make 指令，可以看到，文件均编译成功：

```
osaq@LAPTOP-QS28APKV:~/oslab/lab3$ make
+ cc kern/init/entry.S
+ cc kern/init/init.c
+ cc kern/libs/stdio.c
+ cc kern/debug/panic.c
+ cc kern/debug/kdebug.c
+ cc kern/debug/kmonitor.c
+ cc kern/driver/dtb.c
+ cc kern/driver/clock.c
+ cc kern/driver/console.c
+ cc kern/driver/intr.c
+ cc kern/trap/trap.c
+ cc kern/trap/trapentry.S
+ cc kern/mm/pmm.c
+ cc kern/mm/default_pmm.c
+ cc kern/mm/best_fit_pmm.c
+ cc libs/string.c
+ cc libs/printfmt.c
+ cc libs/readline.c
+ cc libs/sbi.c
+ ld bin/kernel
riscv64-unknown-elf-objcopy bin/kernel --strip-all -O binary bin/ucore.img
```

接着，执行 `make qemu` 命令，编译代码并运行内核，实现效果如图所示。（这里的前面 OpenSBI 启动信息部分与上面是一样的，就不进行展示了，我们主要看后面的测试相关结果）

```
(THU.CST) os is loading ...
Special kernel symbols:
  entry 0xffffffffc0200054 (virtual)
  etext 0xffffffffc0201fda (virtual)
  edata 0xffffffffc0207028 (virtual)
  end    0xffffffffc02074a0 (virtual)
Kernel executable memory footprint: 30KB
memory management: default_pmm_manager
physical memory map:
  memory: 0x0000000008000000, [0x0000000008000000, 0x0000000087fffffff].
check_alloc_page() succeeded!
satp virtual address: 0xffffffffc0206000
satp physical address: 0x00000000080206000
++ setup timer interrupts
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks

--- Triggering illegal instruction test ---
sbi_emulate_csr_read: hartid0: invalid csr_num=0x302
Exception: Illegal instruction
Illegal instruction caught at 0xc0200b58

--- Triggering ebreak test ---
Exception type: breakpoint
ebreak caught at 0xc0200b68
```

我们可以看到，在完成 10 次时钟中断（即打印 10 次“100 ticks”）后，内核自动调用了 `test_exceptions()` 进行异常测试。首先是非法指令异常的测试（--- Triggering illegal instruction test ---），测试函数中试图执行指令 `mret`，但由于此时 CPU 处于 S 模式，而 `mret` 只能在 M 模式执行，因此检查出非法指令异常，OpenSBI 模拟器报告了 `sbi_emulate_csr_read: hartid0: invalid csr_num=0x302`，说明尝试访问特权级受限的寄存器失败，触发了非法指令异常。异常被内核的 `exception_handler()` 捕获，输出“Exception: Illegal instruction”及触发地址 `0xc0200b58`。随后内核将 `tf->epc += 4` 跳过该非法指令，使系统能继续执行而非死循环。

然后是断点异常的测试（--- Triggering ebreak test ---），即执行 `ebreak` 指令，它会主动触发断点异常，用于调试或测试陷入机制。结果表明，异常正确被同一个 `exception_handler()` 捕获，对应 `CAUSE_BREAKPOINT` 分支，输出“Exception type: breakpoint”和异常地址 `0xc0200b68`。最后 `tf->epc` 被加 2 跳过触发点，使得系统能够平稳返回并继续运行。

整体结果表明，内核异常处理机制（异常类型输出、异常指令地址输出、`tf->epc` 修正与返回）工作正常，`mret` 与 `ebreak` 异常均被正确识别和恢复。

四、讨论

实验知识点与 OS 原理知识点对应关系

实验中的重要知识点	对应的 OS 原理知识点	含义、关系与差异理解
sscratch 寄存器的中转逻辑 (csw sscratch, sp/csw s0, sscratch, x0)	上下文切换中的关键状态暂存	含义：实验：通过 sscratch 临时寄存中断前的栈指针（旧 sp），避免保存其他寄存器时被新 sp 覆盖；原理：上下文切换需暂存关键状态（如栈指针、程序计数器），确保切换后可恢复原执行流。关系：sscratch 的使用是关键状态暂存原理在 RISC-V 硬件上的具体实现。差异：原理侧重需暂存状态的抽象概念，实验明确了硬件寄存器（sscratch）的中转细节与时机（先暂存再取回）。
SAVE_ALL/RESTORE_ALL 宏（保存 / 恢复通用寄存器与 CSR）	中断上下文保护与恢复机制	含义：实验：通过汇编指令开辟栈空间，按固定偏移保存 x0-x31、sepc、scause 等，构建 struct trapframe；原理：中断发生后需完整保存 CPU 状态（寄存器、程序地址、中断原因），处理完成后恢复以继续原程序执行。关系：宏定义是上下文保护原理的代码落地，struct trapframe 是原理中上下文数据结构的实例。差异：原理仅强调需保存所有状态，实验需考虑硬件特性（如栈向低地址生长、CSR 的读写指令 csw/csw）。
时钟中断处理（IRQ_S_TIMER 分支：clock_set_next_event/ticks 计数）	中断驱动的系统定时与调度触发	含义：实验：周期性响应时钟中断，更新计数并触发打印 / 关机，通过 clock_set_next_event 设置下一次中断；原理：时钟中断是系统的核心，为定时任务、进程调度提供触发时机。关系：实验的时钟中断处理是中断驱动定时原理的最小验证。差异：原理延伸至调度触发（如时间片耗尽后切换进程），实验仅实现基础计数与动作触发，未涉及进程调度。
异常类型分发（trap_dispatch 区分中断 / 异常，调用对应 handler）	中断与异常的分类处理机制	含义：实验：通过 tf->cause 的正负判断中断（负数） / 异常（非负数），分发至 interrupt_handler/exception_handler；原理：OS 需按事件类型（如时钟中断、非法指令异常）分类处理，避免统一逻辑无法适配不同场景。关系：trap_dispatch 是分类处理机制的代码实现，handler 函数对应原理中的事件处理器。差异：原理侧重分类的必要性，实验明确了硬件寄存器（scause）的判断逻辑与分发流程。

实验中的重要知识点	对应的 OS 原理知识点	含义、关系与差异理解
断点 / 非法指令异常处理 (CAUSE_BREAKPOINT/CAUSE_ILLEGAL_INSTRUCTION 分支)	异常捕获与故障恢复机制	含义：实验：输出异常类型与地址，通过 <code>tf->epc += 4/2</code> 跳过错误指令，避免重复触发；原理：对程序错误（如非法指令、断点）需捕获并处理，最小化故障影响（如跳过错误指令、终止进程）。关系：实验的异常分支是故障恢复机制的简化实现（跳过错误指令以继续执行）。差异：原理支持复杂恢复策略（如内存页缺失的换页），实验仅处理简单的指令跳过，未涉及高级恢复逻辑。

OS 原理中重要但实验未涉及的知识点

1、基于时钟中断的进程调度机制

时钟中断在 OS 原理中是抢占式调度的核心触发源：当进程运行时间超过分配的时间片，时钟中断会强制触发上下文切换，将 CPU 资源分配给就绪队列中的其他进程（如时间片轮转调度）。这一机制是多道程序设计的基础，直接决定系统的资源利用率与响应速度。

实验中虽实现了时钟中断的响应与计数，但未涉及进程就绪队列管理、上下文切换的完整流程（进程间切换）、时间片分配与超时判断等核心逻辑，仅通过 ticks 计数触发打印 / 关机，未体现时钟中断在资源分配中的核心价值。

2、中断优先级与中断屏蔽机制

OS 原理中，中断优先级是处理多中断并发的关键：不同类型的中断（如电源故障中断、磁盘 I/O 中断、时钟中断）具有不同优先级，高优先级中断可打断低优先级中断的处理，确保关键事件优先响应；中断屏蔽则通过屏蔽低优先级中断，保证临界区代码的原子性，避免并发导致的数据错乱。

实验中仅处理单一类型的时钟中断与少量异常，未涉及多中断并发场景，因此未实现优先级判断逻辑与中断屏蔽操作，无法体现 OS 对多中断的有序管理能力。

3、中断驱动的 I/O 控制方式

中断是 I/O 设备与 CPU 交互的核心方式：当 I/O 设备（如磁盘、键盘）完成数据读写后，会通过中断通知 CPU，CPU 无需轮询等待 I/O 完成，可在中断触发后再处理数据，如读取键盘输入、处理磁盘数据。这一方式大幅提升了 CPU 与 I/O 设备的并行效率，是现代 OS 的 I/O 管理基础。

实验中未涉及任何外设 I/O 驱动（如键盘、显示器）的实现，仅通过 `sbi_shutdown` 调用底层接口关机，未体现中断驱动 I/O 的完整流程，无法验证中断在 I/O 管理中的实际作用。

五、实验结论及心得体会

实验结论

本次实验我们成功实现了 RISC-V 架构下 ucore 内核的中断与异常处理功能，核心目标均达成：

中断机制落地：通过初始化 stvec 寄存器指向中断入口_alltraps、配置 sscratch 完成栈指针暂存，构建了完整的中断响应链路，时钟中断可周期性触发，每 100 次中断打印计数信息，累计 10 次后正常触发关机流程。

异常处理生效：实现了非法指令 (mret) 与断点 (ebreak) 异常的捕获与处理，能准确输出异常类型及指令地址，并通过调整 sepc 寄存器跳过错指令，确保程序可继续执行，无重复异常或崩溃问题。

上下文管理可靠：借助 SAVE_ALL/RESTORE_ALL 宏与 struct trapframe，完成通用寄存器与 sepc、scause 等关键 CSR 的保存与恢复，保障中断 / 异常处理前后程序执行流的无缝衔接。

心得体会

1、在实验前我们对上下文保护的理解仅停留在保存寄存器的抽象层面，实操中我们明白了 RISC-V 的硬件约束，比如用 sscratch 中转栈指针避免覆盖，按栈生长方向（低地址）开辟 struct trapframe 空间，通过 csrr/csrw 指令操作 CSR 等。这些说明，OS 原理的每一个逻辑，都必须依托具体硬件的寄存器设计与指令规范实现。

2、从时钟中断的触发→保存上下文→计数处理→设置下次中断，到异常的 捕获→类型判断→地址输出→调整 sepc，每一步都环环相扣。例如，若忘记调用 clock_set_next_event，时钟中断会仅触发一次；若 sepc 偏移计算错误，程序会陷入死循环。这说明 OS 中断系统的可靠性，依赖于硬件触发 - 软件处理 - 状态重置全流程的无疏漏设计。

3、在实验中我们仅实现了单一时钟中断与两类异常的处理，未涉及中断优先级、进程调度触发、I/O 中断等原理中的核心延伸点。但正是这种简化，让我们能聚焦上下文保存、中断分发等基础逻辑，理解其作为 OS 事件响应核心的本质。